

Ingeniería del Software de Componentes y Sistemas Distribuidos

PRUEBA PRÁCTICA – Práctica 1

Presentación

En la PEC 1 se han analizado diferentes estilos arquitectónicos y los criterios para seleccionar cuál o cuáles de ellos son los más adecuados para guiar el desarrollo de una solución.

En esta Práctica 1 veremos cómo diseñar una solución basada en una arquitectura de microservicios, usando modelos de representación y patrones de diseño.

La Práctica 1 abarca los contenidos del libro "Microservices Patterns" (ver en el apartado Recursos los capítulos que hay que estudiar).

Competencias

En la Práctica 1 se trabaja la siguiente competencia del Grado en Ingeniería Informática:

- Capacidad de analizar un problema en el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y resolverlo.

Objetivos

Los objetivos de la Práctica 1 son:

- Identificar operaciones que un sistema tiene que proporcionar.
- Proponer el diseño de una arquitectura y su descomposición en microservicios.
- Diseñar de forma detallada un microservicio.
- Evaluar y justificar cómo resolver una problemática concreta en el diseño de microservicios (escoger un patrón, un estilo de comunicación o resolver la transaccionalidad).

Descripción de la Práctica

Después de una nueva ronda de reuniones con el grupo inversor, están muy satisfechos del estilo arquitectónico basado en microservicios que hemos propuesto para **Photo&Film4You** en la PEC1. Sin embargo, nos han pedido añadir nuevos requisitos y alguna puntualización/restricción al catálogo de alquiler de material audiovisual:

- Se deben considerar los estados “planificado”, “cancelado” y “devuelto” para los alquileres de material, sigue quedando fuera del ámbito del proyecto la gestión de los casos en que el cliente no retorna el material alquilado o lo retorna defectuoso.
- Las categorías y subcategorías no se pueden modificar ni dar de baja.
- De los productos, siempre se puede actualizar su descripción, pero sólo se puede modificar el precio de alquiler por día en caso que no tengan unidades de producto comprometidas en ningún alquiler.
- Se pide añadir un nuevo servicio de adquisición por parte de la empresa de material de segunda mano, con el siguiente flujo:
 - Un usuario registrado en el sistema envía una oferta de un producto que dispone de segunda mano. Esta petición queda en estado pendiente de validación en la plataforma.
 - Un administrador de **Photo&Film4You** valida la petición y si el artículo se encuentra en buenas condiciones se incorpora en el estoc de unidades disponible en el alquiler para es producto.
- Para simplificar este caso de uso:
 - La gestión de compra de producto de segunda mano queda fuera del alcance del proyecto: precio de la oferta, pago al usuario, etc.
 - No se requiere ninguna distinción a efecto del coste del alquiler por un día de una unidad de producto adquirida de este nuevo mercado de segunda mano respecto a otra unidad del producto ya existente en el inventario. Se supone que pasa una revisión antes de incorporar el artículo en el sistema.
 - Un usuario sólo puede proponer una oferta de artículo de segunda mano sobre una categoría y subcategoría de productos ya existentes en el catálogo.
- Los productos solo se pueden dar de baja cuando no tengan unidades de producto comprometidas en ningún alquiler y esta operación dará de baja el producto y todas sus unidades de producto. De cara al usuario los productos y las unidades de producto, se dan de baja pero realmente no se eliminan de la base de datos si no que quedan en un estado de “no activos”
- Las unidades de los productos solo podrán pasar a estado “no operativo” si no están comprometidas con ningún alquiler.
- Tecnológicamente, nos piden que la persistencia de la plataforma esté soportada por JPA y MySQL.

Ejercicio 1 (3 puntos)

En base a todos los requisitos establecidos en la especificación del caso **Photo&Film4You**, especificar la arquitectura de microservicios de la aplicación indicando:

1. Operaciones del sistema (1 punto). Usando el patrón CQRS, separar los **comandos** de las **consultas**:

Para cada uno de los **comandos** especificar:

- Actor
- Caso de uso
- Nombre del comando/*command*
- Descripción

Para cada una de las **consultas** especificar:

- Actor
- Caso de uso
- Nombre de la consulta/*query*
- Descripción

NOTA: Se debe usar una nomenclatura tipo “Lower Camel Case” (<https://wiki.c2.com/?LowerCamelCase>) para los nombres de los comandos y consultas.

2. Especificación de los comandos (1 punto). Para cada comando, especificar sus parámetros, el valor de retorno, las precondiciones y sus postcondiciones.

3. Modelo general de dominio (1 punto). Diagrama a alto nivel del modelo de dominio.

NOTA: Tenéis un ejemplo del diagrama solicitado en el libro "Microservices patterns", capítulo 2: "Decomposition strategies", figura 2.7: "The key classes in the FTGO domain model".

Solución

1. Operaciones del sistema

Los clientes web se comunicarán con HTTP, pero puede haber clientes que se comuniquen con mensajería (colas de mensajes). Para no estar ligados pues a un protocolo específico, hacemos primero esta abstracción en operaciones del sistema para representar las diferentes peticiones que puede recibir nuestro sistema.

Las operaciones del sistema las dividimos, usando el patrón CQRS, en:

- Comandos. Operaciones del sistema que crean, actualizan o borran datos

- Consultas. Operaciones del sistema que leen o consultan datos

Comandos (Commands):

Actor	Caso de uso	Comando	Descripción
Administrador	Dar de alta una sección/subsección	createCategory()	Crear una sección/subsección en el catálogo
Administrador	Dar de alta un producto	createProduct()	Crear un producto dentro del catálogo
Administrador	Modificar la descripción de un producto	updateProductDescription()	Modificar la descripción de un producto dentro del catálogo
Administrador	Modificar el precio diario de alquiler de un producto	updateProductDailyPrice()	Modificar el precio diario de alquiler de un producto dentro del catálogo
Administrador	Dar de baja un producto	deleteProduct()	Da de baja un producto del catálogo
Administrador	Dar de alta una unidad de un producto	createItem()	Crear una unidad de un producto dentro del catálogo
Administrador	Establece una unidad de un producto como no operativa.	setNotOperationalItem()	Establece una unidad de un producto como no operativa y no puede ser alquilada.
Administrador	Establece una unidad de un producto como operativa.	setOperationalItem()	Establece una unidad de un producto como operativa y ya puede volver a ser alquilada.
Administrador	Valida una oferta de un producto de segunda mano	evaluateOffer()	Actualiza el estado de una oferta de un producto de segunda mano
Usuario no registrado	Registrarse en el sistema	signUp()	Registrarse en el sistema.
Usuario	Modificar los datos personales	updateUserProfile()	Actualizar los datos personales

Usuario	Crear una nueva cesta	addCart()	Crea una nueva cesta con el rango de fechas y la unidad de producto especificadas.
Usuario	Añadir la unidad a la cesta	addItemToCart()	Añade la unidad del producto a la cesta para el rango de fechas especificadas.
Usuario	Suprimir la unidad de la cesta	deleteItemFromCart()	Suprime la unidad del producto a la cesta.
Usuario	Dar de alta una petición de alquiler	checkoutRent()	Crea una petición de alquiler con el contenido de la cesta.
Usuario	Anular una petición de alquiler	cancelRent()	Anula la petición de alquiler.
Usuario	Dar de alta una alerta	createAlert()	Crea una alerta de disponibilidad de un producto en un rango de fechas.
Usuario	Dar de baja una alerta	dropAlert()	Da de baja una alerta de disponibilidad de un producto en un rango de fechas.
Usuario	Dar de alta una oferta de un producto de segunda mano	addOffer()	Da de alta una oferta de un producto de segunda mano

Consultas (Queries):

Actor	Caso de uso	Query	Descripción
Usuario no registrado	Consultar las secciones / subsecciones disponibles	findCategoriesByCriteria()	Consultar las secciones y subsecciones disponibles en el catálogo por nombre, descripción o categoría padre.
Usuario no	Buscar productos	findProductsByCriteria()	Buscar productos

registrado	por criterio		por diferentes criterios (categoria, nombre, descripción, marca, modelo)
Usuario no registrado	Consultar la información de un producto	getProductDetails()	Obtener información de un producto, incluye las unidades de producto.
Usuario no registrado	Consultar la información de una unidad	getItemDetails()	Obtener información de una unidad de un producto, incluye las peticiones de alquiler activas.
Usuario no registrado	Consultar las unidades disponibles en un rango de fechas.	findRentAvailableItems()	Consultar las unidades de un producto disponibles para su alquiler en un rango especificado de fechas.
Usuario	Consultar los datos personales	getUserProfile()	Recuperar los datos personales.
Usuario	Consultar la información de la cesta.	getCart()	Consultar el contenido de la cesta.
Usuario	Consulta las alertas de disponibilidad de un usuario.	findAlertsByUser()	Consulta las alertas de disponibilidad de un producto en un rango de fechas para un usuario.
Usuario	Consultar la información de una alerta de disponibilidad.	getAlert()	Consulta la información de una alerta de disponibilidad.
Usuario	Consultar las peticiones de alquiler de un usuario	findRentsByUser()	Consulta las peticiones de alquiler de un usuario.

Usuario	Consulta las ofertas de un usuario	findOffersByUser()	Consulta las ofertas de productos de segunda mano de un usuario
Usuario / Administrador	Consultar la información de la petición de alquiler.	getRent()	Consultar la información de una petición de alquiler.

*NOTA: No se representa ninguna operación del sistema relacionada con el login, la autenticación es gestionada con el componente **API Gateway**, por tanto, no se considera una necesidad real del dominio. Se consideran correctas las soluciones que hayan representado operaciones de login.*

2. Especificación de los comandos

- **createCategory**

Operación: createCategory(name, description, parent)

Retorno: categoryId

Precondiciones:

- No existe previamente ninguna categoría con ese nombre.
- Si se especifica la categoría padre ésta debe existir.

Postcondiciones:

- La categoría se ha creado.
- Si se especifica la categoría padre se crea una relación padre/hijo entre ambas.

- **createProduct**

Operación: createProduct(categoryId, name, description, brand, model, dailyPrice)

Retorno: productId

Precondiciones:

- La categoría existe previamente.
- No existe ningún producto previamente con ese nombre.

Postcondiciones:

- El producto se crea sin ninguna unidad como producto dentro de la categoría especificada.

- **updateProductDescription**

Operación: updateProductDescription (productId, newDescription)

Retorno: True/False

Precondiciones:

- El producto existe previamente.

Postcondiciones:

- La descripción del producto se ha actualizado con el valor indicado.

- **updateProductDailyPrice**

Operación: updateProductDailyPrice (productId, newDailyPrice)

Retorno: True/False

Precondiciones:

- El producto existe previamente.
- El producto no tiene ninguna unidad de producto comprometida en alquileres.

Postcondiciones:

- El precio diario del producto se ha actualizado con el valor indicado.

- **deleteProduct**

Operación: deleteProduct(productId)

Retorno: True/False

Precondiciones:

- El producto existe previamente.
- El producto no tiene ninguna unidad de producto comprometida en alquileres.

Postcondiciones:

- Se actualiza el estado **INACTIVE** del producto y todas sus unidades de producto asociadas.

- **createItem**

Operación: createItem(productId, serialNumber)

Retorno: itemId

Precondiciones:

- El producto existe previamente.
- No existe ninguna unidad con ese número de serie.

Postcondiciones:

- Se crea una unidad del producto en estado **OPERATIONAL** y sin ninguna reserva de alquiler.

- Se notifica por correo electrónico de la nueva unidad disponible a todos los usuarios que tengan una alerta de disponibilidad sobre ese producto, independientemente de las fechas de la alerta.

- **setNotOperationalItem**
 Operación: setNotOperationalItem(itemId)
 Retorno: True/False
 Precondiciones:
 - La unidad de producto existe previamente y está en estado OPERATIONAL.
 - La unidad de producto no está comprometida con ningún alquiler.
 Postcondiciones:
 - La unidad de producto pasa a estado NOT_OPERATIONAL y no puede ser alquilada.

- **setOperationalItem**
 Operación: setOperationalItem(itemId)
 Retorno: True/False
 Precondiciones:
 - La unidad de producto existe previamente y está en estado NOT_OPERATIONAL.
 Postcondiciones:
 - La unidad de producto pasa a estado OPERATIONAL y puede ser alquilada.

- **EvaluateOffer**
 Operación: evaluateOffer(offerId, date, state)
 Retorno: state (el nuevo estado de la oferta)
 Precondiciones:
 - La oferta por el producto de segunda mano existe previamente.
 Postcondiciones:
 - Si se acepta la oferta se actualiza el nuevo estado ACCEPTED y la fecha en la que se ha producido la aceptación de la oferta. La aceptación de la oferta implica incorporar un nuevo item en el catálogo.. Si no se acepta la oferta se actualiza el nuevo estado y la fecha y el nuevo estado será REJECTED

- **signUp**
 Operación: signUp(email, password, fullName, phoneNumber)
 Retorno: True/False

Precondiciones:

- No existe ningún usuario identificado con ese email en el sistema.

Postcondiciones:

- El usuario queda registrado al sistema y está operativo.

- **updateUserProfile**

Operación: - updateUserProfile(email, password, fullName, phoneNumber)

Retorno: True/False

Precondiciones:

- El usuario identificado por el email indicado existe previamente.

Postcondiciones:

- Los datos para fullName y phoneNumber del perfil del usuario identificado por el email indicado se han actualizado con los valores indicados.

- **createCart**

Operación: createCart(email, itemId, from, to)

Retorno: cartId

Precondiciones:

- El usuario identificado por el email indicado existe previamente.
- La unidad de producto existe y está disponible para el alquiler en las fechas especificadas.

Postcondiciones:

- Si el usuario no tenía ninguna cesta se crea una nueva con las fechas de inicio y fin del alquiler especificadas.
- Si el usuario tenía una cesta con fechas diferentes se elimina la cesta anterior y se crea una nueva con las fechas de inicio y fin del alquiler especificadas.
- La unidad de producto se ha añadido a la cesta del usuario.
- El total del importe con el precio de alquiler por día del producto al que pertenece la unidad multiplicado por el número de días de alquiler.

- **addItemToCart**

Operación: addItemToCart(email, itemId, cartId)

Retorno: cartId

Precondiciones:

- El usuario identificado por el email indicado existe previamente.
- La cesta existe.
- La unidad de producto existe y está disponible para el alquiler en las fechas especificadas.

Postcondiciones:

- La unidad de producto se ha añadido a la cesta del usuario.
- El total del importe se ha actualizado sumando al precio total el precio de alquiler por día del producto al que pertenece la unidad multiplicado por el número de días de alquiler.

- **deleteltemFromCart**

Operación: deleteltemFromCart (email, itemId, cartId)

Retorno: True/False

Precondiciones:

- El usuario identificado por el email indicado existe previamente.
- La cesta existe.
- La unidad de producto existe y está en la cesta del usuario.

Postcondiciones:

- La unidad de producto se ha eliminado de la cesta del usuario.
- El total del importe se ha actualizado restando al precio total el precio de alquiler por día del producto al que pertenece la unidad multiplicado por el número de días de alquiler.

- **checkoutRent**

Operación: checkoutRent(cartId)

Retorno: orderId

Precondiciones:

- La cesta existe.

Postcondiciones:

- Se crea una petición de alquiler para las fechas y las unidades de producto que contiene la cesta del usuario.
- El pago por el importe del total de la cesta se ha autorizado por parte del sistema externo de pagos.
- La petición de alquiler queda en estado SCHEDULED y las unidades de producto de la cesta pasan a tener una reserva de alquiler en las fechas especificadas en la misma.

- **cancelRent**

Operación: cancelRent(rentId)

Retorno: True/False

Precondiciones:

- La petición de alquiler existe.

Postcondiciones:

- La petición de alquiler queda en estado CANCELLED y las unidades de producto de la petición de alquiler pasan a estar disponibles para las fechas del alquiler cancelado.
- Se notifica por correo electrónico de que las unidades están disponibles a todos los usuarios que tengan una alerta de disponibilidad sobre ese producto en las fechas que tenía la petición de alquiler.

- **createAlert**

Operación: createAlert(email, productId, from, to)

Retorno: alertId

Precondiciones:

- El producto existe previamente.
- El usuario identificado por el email indicado existe previamente.

Postcondiciones:

- Se crea una nueva alerta asociada al usuario identificado por email que será notificado cuando haya unidades del producto identificado por productId disponibles para las fechas indicadas.

- **dropAlert**

Operación: dropAlert(email, alertId)

Retorno: True/False

Precondiciones:

- La alerta identificada con alertId existe previamente y está asociada con el usuario identificado por el email indicado.

Postcondiciones:

- Se elimina del sistema la alerta identificada por alertId y el usuario identificado por email ya no será notificado cuando haya unidades disponibles del producto en las fechas que especificaba la alerta.

- **addOffer**

Operación: addOffer(email, categoryId, productId, , serialNumber)

Retorno: offerId

Precondiciones:

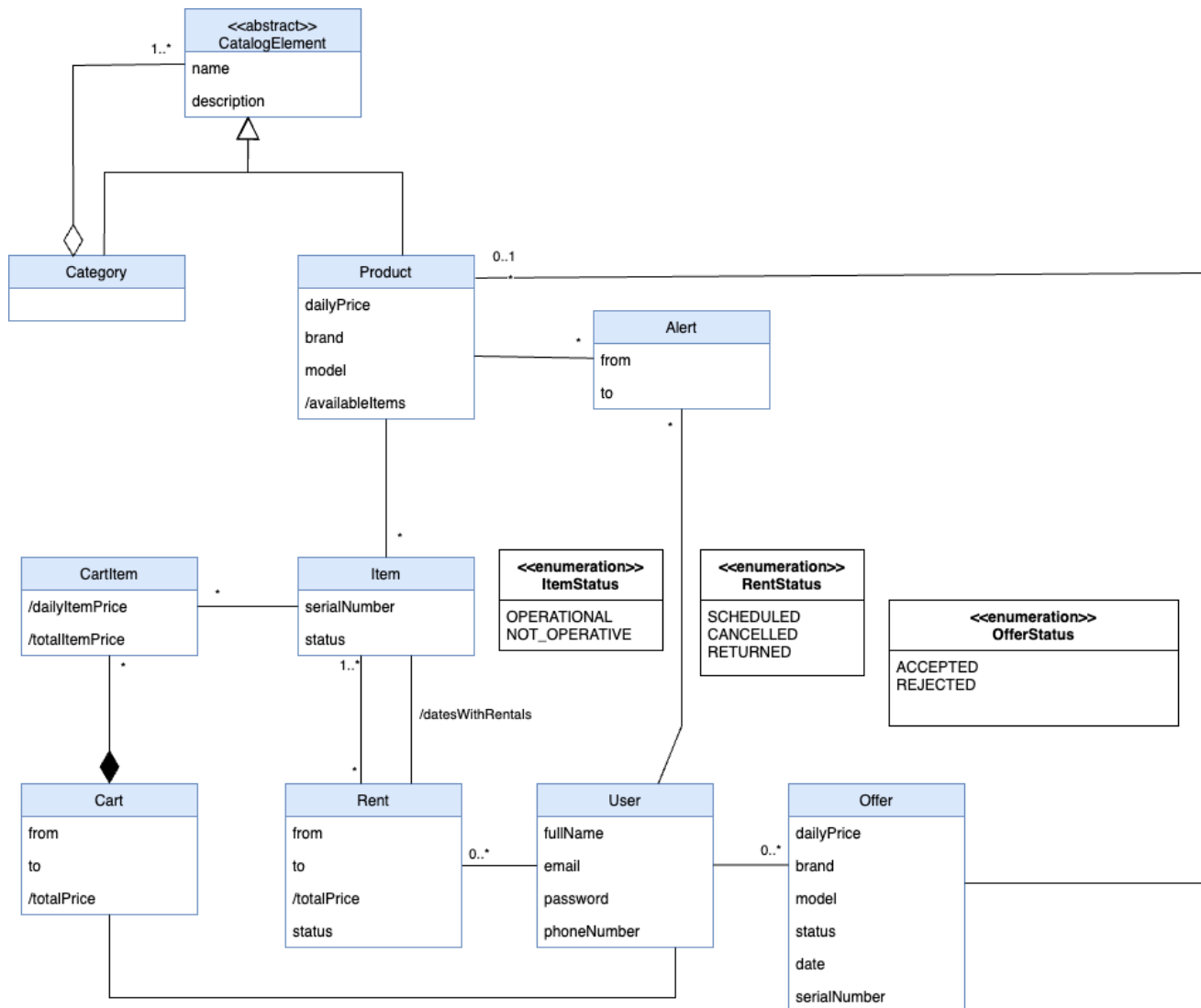
- El usuario identificado por email existe

Postcondiciones:

- Se crea la nueva oferta de producto de segunda mano

3. Modelo de dominio (alto nivel)

El modelo de dominio a alto nivel describe las clases clave usando el vocabulario con el que describimos las operaciones del sistema.



Notas:

- Se considerará correcto el modelado de marcas y modelos de los productos como entidades.
- Se considerará correcto el modelado del catálogo sin usar el patrón de diseño “Composite”

Referencias:

- Libro “Microservices patterns”: Sección 2.2.1 "Creating a high-level domain model".
- Libro “Microservices patterns”: Sección 2.2.1 "Defining system operations".

Ejercicio 2 (2 puntos)

En base a la definición de las operaciones del sistema y el modelo de dominio del ejercicio anterior, realizar la descomposición de los diferentes **microservicios** de la aplicación:

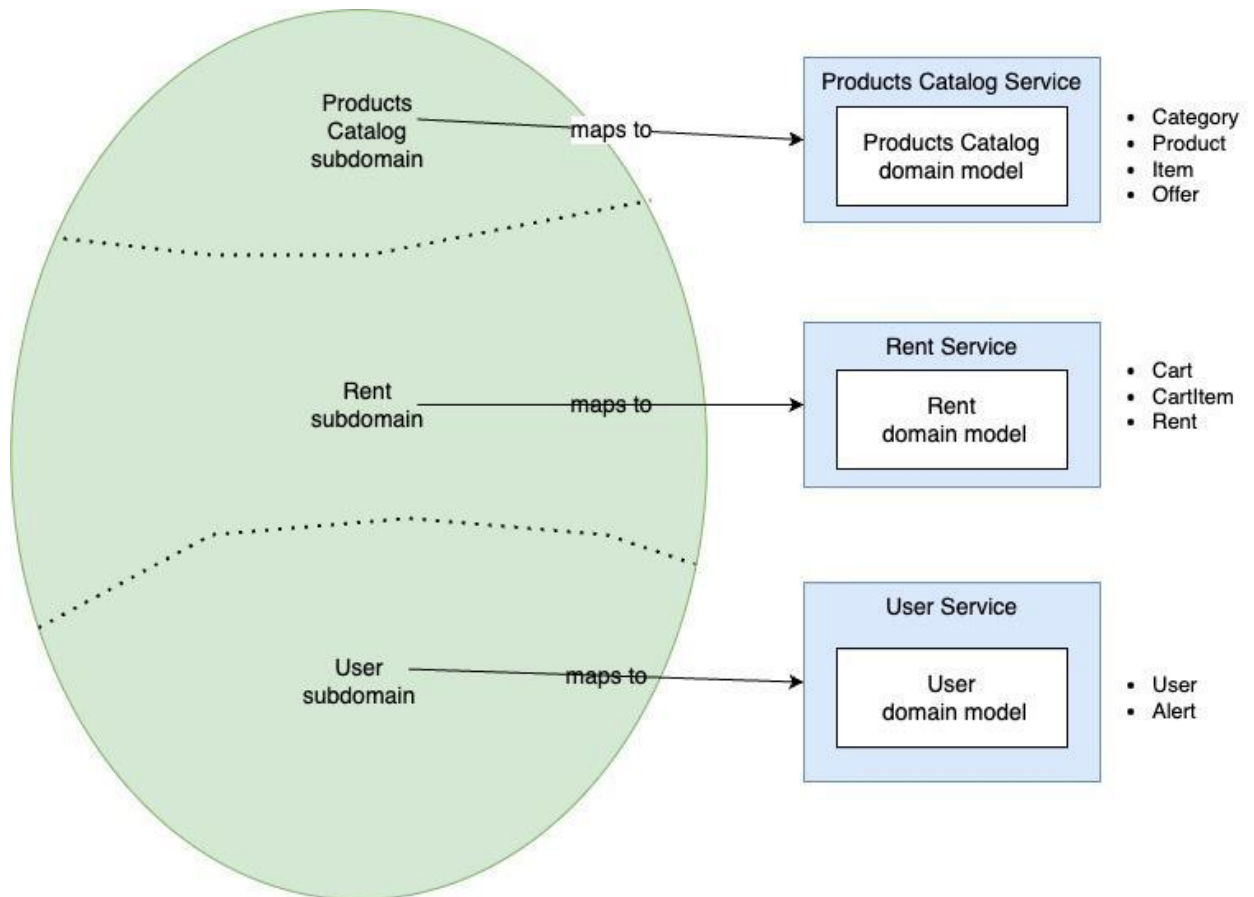
1. Identificar los subdominios y sus **servicios** necesarios, usando una descomposición basada en el patrón de sub-dominios (1 punto).
2. Realizar el **mapeo** entre las operaciones del sistema y los servicios identificados, e identificar las colaboraciones que requieren unos servicios de otros (1 punto).

Solución

1. A continuación, se presenta una posible descomposición del sistema en los subdominios y los **servicios** necesarios, usando una descomposición basada en el patrón de sub-dominios descrita en el capítulo 2, apartado 2.2.3 *“Defining services by applying the Decompose by sub-domain pattern”* del libro de referencia.

Tened en cuenta que esta es una posibilidad entre otras, tal como se comenta en el capítulo 1, apartado 1.5.2 *Drawbacks of the microservice architecture* del libro de referencia:

“One challenge with using the microservice architecture is that there isn’t a concrete, well-defined algorithm for decomposing a system into services. As with much of software development, it’s something of an art”

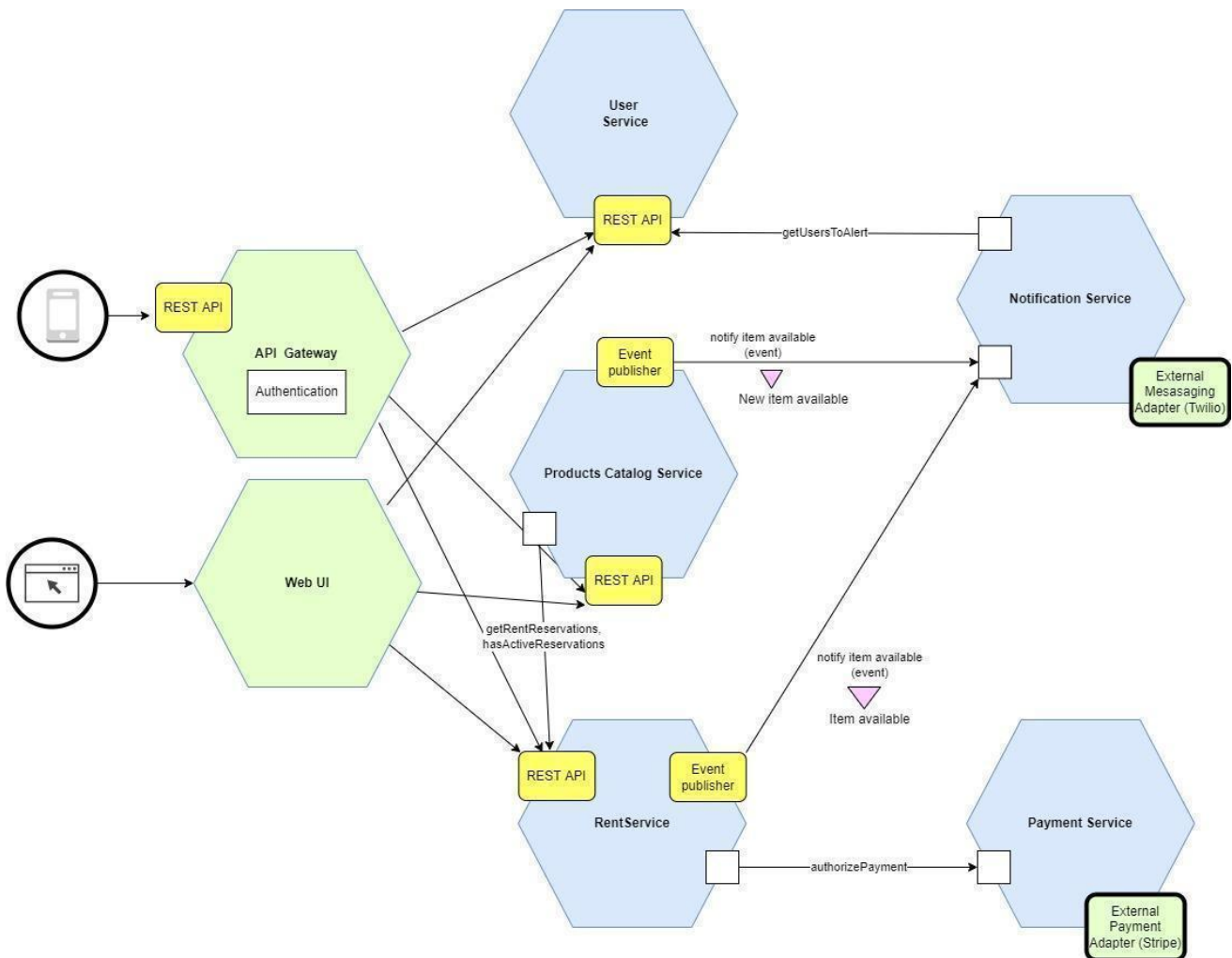


- **Products Catalog Service:** Gestionar el catálogo de material para alquilar y las ofertas de productos de segunda mano..
- **Rent Service:** Gestión de la cesta de compra y las peticiones de alquiler realizadas por los usuarios.
- **User Service:** Gestionar el registro de usuarios y las alertas asociadas.
- **Payment Service:** Gestionar los pagos. No se ha definido ningún subdominio, puesto que no se quiere tener ninguna información del pago.
- **Notification Service:** Servicio transversal de notificaciones, en el caso concreto permitirá notificar a los usuarios con alertas activas de la disponibilidad de algún elemento del catálogo.

2. Para cada servicio identificado, establecer cuáles son las operaciones del sistema que resuelve, e identificar las colaboraciones que requieren otros servicios.

Servicio	Operaciones	Colaboradores
Products Catalog Service	- createCategory() - createProduct() - updateProductDescription() - updateProductDailyPrice() - deleteProduct () - createItem() - setNotOperationalItem() - setOperationalItem() - findCategoriesByCriteria() - findProductsByCriteria() - getProductDetails() - getItemDetails() - findRentAvailableItems() - addOffer() - evaluateOffer() - findOffersByUser()	Rent Service: - getRentReservations() - hasActiveReservations
Rent Service	- addCart() - addItemToCart() - deleteItemFromCart() - getCart() - checkoutRent() - cancelRent() - getRent() - findRentsByUser()	Payment Service: authorizePayment()
User Service	- signUp() - updateUserProfile() - getUserProfile() - createAlert() - dropAlert() - findAlertsByUser() - getAlert() - getUsersToAlert()	-
Notification Service	sendItemAvailable()	User Service: - getUsersToAlert() Products Catalog Service: - getItemDetails()

Payment Service	▪ authorizePayment()	-
-----------------	----------------------	---



Durante la descomposición podemos ver la necesidad de añadir operaciones internas que los microservicios tienen que ofrecer y no habíamos detectado en primera instancia.

Ejemplos:

- *getUsersToAlert*: Operación que ofrece *User Service* para que se puedan recuperar los usuarios a notificar en caso de que se dé de alta alguna unidad o se produzca alguna anulación de petición de alquiler sobre un producto que tenga alguna alerta activa.
- *getRentReservations*: Operación que ofrece *Order Service* para que se puedan recuperar las reservas activas de una unidad de producto.
- *hasActiveReservations*: Operación que ofrece *Order Service* para que se pueda saber si un producto tiene unidades de producto con reservas activas.

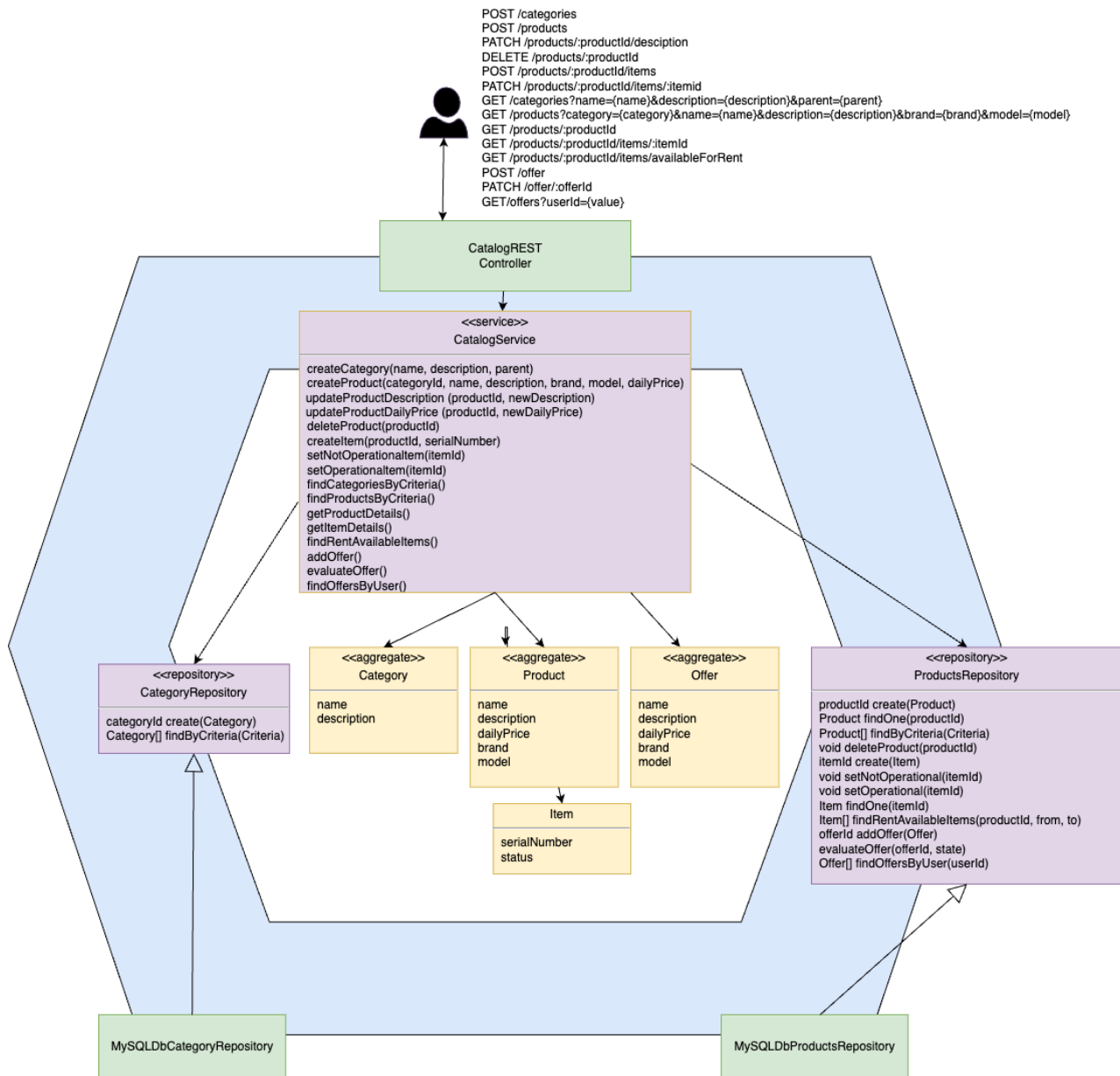
En el diagrama se muestran adaptadores de ejemplo reales que se podrían usar por los servicios de pago y de notificaciones.

Ejercicio 3 (3 puntos)

Partiendo de la descomposición realizada en el ejercicio anterior, detallar el **diseño** del servicio de catálogo de productos, teniendo en cuenta:

- El diseño se tiene que realizar usando una **arquitectura hexagonal**.
- El diagrama tiene que representar los **protocolos de comunicación** con el servicio (REST, etc.), y los **métodos y atributos** de las diferentes clases del servicio (no es necesario indicar los tipos).

Solución



Consideraciones:

- Existen diversas aproximaciones para implementar los filtros/búsquedas en una API RESTful, en la solución se ha usado la aproximación más simple que es hacer peticiones GET pasando los filtros de la búsqueda como parámetros en la URL.

- Toda unidad forma parte de un producto. Por este motivo el agregado es el producto y todas las operaciones que se hacen sobre las unidades se realizan desde el recurso producto (*products*).
- Las operaciones de modificación del precio diario y la descripción del producto se han mapeado como una única operación PATCH sobre el producto. Se considera también correcto el mapeo como dos operaciones POST o PUT específicas.
- Las operaciones que cambian el estado de una unidad se han mapeado como operaciones PATCH sobre la unidad del producto en cuestión. Se considera también correcto el mapeo como dos operaciones POST o PUT específicas.

Fuentes de información para la resolución del ejercicio:

- Libro "Microservices patterns": Capítulo 3, sección "Specifying REST APIs".
- Libro "Microservices patterns": Capítulo 5, sección "Designing a domain model using the DDD aggregate patterns".

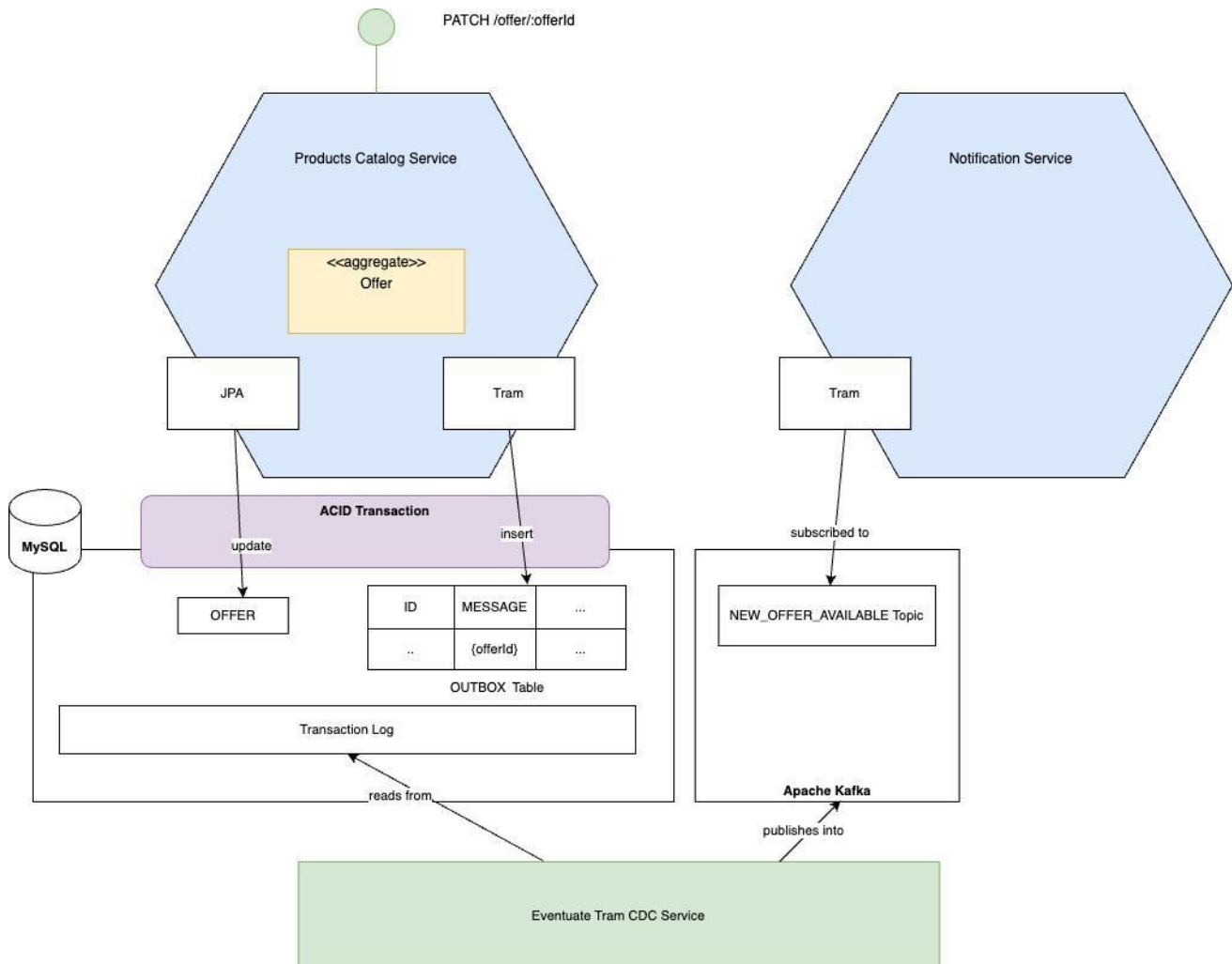
Ejercicio 4 (2 puntos)

Cuando se realiza una validación de una oferta de un producto de segunda mano intervienen diferentes sistemas: envío de mensaje a una cola con envío de correo al usuario que ha realizado la oferta y a otros responsables de la gestión de las ofertas de productos de segunda mano. Queremos asegurar que se realiza de forma atómica.

Es decir, en caso de que por ejemplo falle la actualización de la base de datos del estado de la oferta del producto de segunda mano, no se haría ningún envío del mensaje a la cola, y por tanto que no se envíe ningún correo de notificación.

Representar en un diagrama la solución a este planteamiento, usando el patrón o patrones de microservicios más adecuado/s. Justificar las decisiones tomadas.

Solución



El patrón escogido para resolver la problemática planteada es **Transactional Outbox**:
<https://microservices.io/patterns/data/transactional-outbox.html>

Cuando se modifica el estado de una oferta de producto de segunda mano se debe consultar a los usuarios suscritos a alertas para esa oferta de producto y notificarles la nueva disponibilidad. Esto se puede conseguir mediante el patrón **Transaccional Outbox**, publicando el evento asociado a la nueva disponibilidad en una tabla de mensajes (OUTBOX) y usando un *Polling Pattern Publisher* o *Transactional Log Tailing Pattern* para publicar los eventos de forma transaccional.

Esta solución tiene en cuenta la premisa de que un error en la notificación a uno de los usuarios suscritos no impide la actualización de la nueva oferta de producto, separando las transacciones de

"modificación de oferta+alta de notificación-outbox" de la del "envío de la notificación+marcado de outbox como notificación realizada" en los dos servicios "Product Catalog" y "Notification"

Fuentes de información para la resolución del ejercicio:

- Libro "Microservices patterns": Capítulo 3, sección "Communicating using the Asynchronous messaging".
- Libro "Microservices patterns": Capítulo 5.3, sección "Publishing domain events".

Recursos

Recursos Básicos

- Libro "Microservices Patterns" (Richardson):
 - o Chapter 1. "Escaping monolithic hell".
 - o Chapter 2: "Decomposition strategies".
 - o Chapter 3: "Interprocess communication in a microservice architecture".
 - o Chapter 5: "Designing business logic in a microservice architecture".
 - o Chapter 7 "Implementing queries in a microservice architecture".

Recursos Complementarios

- Sitio web: "A pattern language for microservices". <https://microservices.io/patterns/index.html>

Criterios de evaluación

- En esta actividad **no está permitido el uso de herramientas de inteligencia artificial** y se tiene que resolver de forma **estrictamente individual**. En caso de detectar irregularidades se penalizará la actividad con una D como nota. En el plan docente y en el [web sobre integridad académica y plagio de la UOC](#) encontraréis información sobre qué se considera conducta irregular en la evaluación y las consecuencias que puede tener.
- El peso de cada ejercicio está indicado en el enunciado.
- Los criterios de evaluación de cada ejercicio están indicados en el anexo.
- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Se debe entregar un único documento PDF con las respuestas a todos los ejercicios. El nombre del archivo tiene que ser: *ApellidosNombre_ISCSD_PRAC1.pdf*.

Este documento se entregará en el espacio “Entrega de la PRAC1” del aula antes de las **23:59 horas del día 19 de abril de 2024**. No se aceptarán entregas fuera de plazo.

Anexo

Criterios de evaluación

	Excelente (10)	Notable (7)	Suficiente (5)	Insuficiente (2,5)	Sin respuesta / Todo mal (0)
Ej. 1.1	Enumerar más de 20 operaciones del sistema de forma correcta (1)	Enumerar entre 10 y 20 operaciones del sistema de forma correcta (0,75)	Enumerar menos de 10 operaciones del sistema de forma correcta (0,5)	Otras (operaciones incorrectas, formato incorrecto, etc.) (0,25)	0
Ej. 1.2	Especificar más de 15 comandos de forma correcta (1)	Especificar entre 5 y 15 comandos de forma correcta (0,75)	Especificar menos de 5 comandos de forma correcta (0,5)	Otras (comandos incorrectos, formato incorrecto, etc.) (0,25)	0
Ej. 1.3	Modelo de dominio correcto con la mayor parte de entidades y relaciones (1)	Modelo de dominio incompleto, con pocas entidades y/o relaciones, pero la especificación correcta (0,75)	Modelo de dominio incorrecto LEVE (sin atributos, con operaciones, etc.) (0,5)	Modelo de dominio incorrecto GRAVE (entidades del dominio de la solución y no del problema, sin relaciones, etc.) (0,25)	0
Ej. 2.1	Identificación correcta y diagrama bien hecho (1)	Identificación no del todo correcto (demasiado o pocos	Identificación incorrecta o diagrama mal hecho (0,5)	Identificación incorrecta diagrama mal hecho (0,25)	0

		subdominios más de un servicio por subdominio, etc...) y diagrama bien t (0,75)			
Ej. 2.2	Mapeo de más de 20 operaciones de forma correcta con las colaboraciones (1)	Mapeo de más de 20 operaciones de forma correcta sin las colaboraciones o con colaboraciones incorrectas (0,75)	Algunas operaciones correctas con algunas colaboraciones bien (0,5)	La mayoría de operaciones incorrectas con la mayoría de colaboraciones mal. (0,25)	0
Ej. 3	Incorpora diagrama del servicio del catálogo de productos con arquitectura hexagonal + Describe correctamente los protocolos de comunicación (verbos HTTP) + Incorpora correctamente los siguientes elementos/clases: controladores, servicios, agregados, repositorios + Muestra las relaciones entre los diferentes elementos + Cada elemento de los anteriores tiene informados sus	Incorpora diagrama del servicio del catálogo de productos con arquitectura hexagonal pero no es del todo correcto + Error en alguno de los protocolos de comunicación (verbos HTTP) + Incorpora con algún error los siguientes elementos/clases: controladores, servicios, agregados, repositorios + Muestra las relaciones entre los distintos elementos de forma incompleta + Algunos elementos de	Incorpora diagrama del servicio del catálogo de productos con arquitectura hexagonal, pero, en buena parte, incorrecto. Error en la mayoría de protocolos de comunicación (verbos HTTP) + Incorpora muchos errores en los siguientes elementos/clases: controladores, servicios, agregados, repositorios + Muestra las relaciones entre los diferentes elementos de muchas carencias + No	Otros: sin diagrama, diagrama incorrecto 100%, arquitectura no hexagonal, etc.+ Sin protocolos de comunicación (verbos HTTP) o todos mal + No incorporar dichos elementos/clases + No se muestran relaciones entre los diferentes elementos + No se muestran atributos ni operaciones sobre los elementos (0,75)	0

	atributos y operaciones (3)	los anteriores tienen informados sus atributos y operaciones (1,75)	se muestran atributos ni operaciones sobre los elementos + La mayoría de elementos de los anteriores tienen informados sus atributos y operaciones (1,5)		
Ej. 4	Elección, justificación y diagrama correctos (2)	Elección y justificación correcta, pero sin diagrama o con diagrama incorrecto o elección incorrecta con justificación y diagrama correctas (1,5)	Elección incorrecta con diagrama correcto, pero sin justificación + Elección incorrecta con justificación correcta, pero sin diagrama (1)	Elección incorrecta con diagrama incorrecto + Elección incorrecta con justificación incorrecta (0,50)	0